SRI Research Directions

Research Projects in Adversarial Machine Learning

Summer Research Internship

July 2026

Abstract

This document presents two research directions in adversarial machine learning. The first explores using zero-knowledge proofs for private certified-robustness auditing, enabling a model owner to prove certified accuracy over a committed test set without revealing model weights, test inputs, labels, or per-sample certificates. The second investigates whether reinforcement learning-based adversarial attack policies transfer across neural network architectures. Both directions address open questions with practical security implications.

Contents

Part I
Zero-Knowledge Proofs for Private Certified Robustness

1 Introduction

1.1 The Problem: Robustness Auditing with Private Assets

Neural networks are increasingly deployed in safety-critical systems: autonomous vehicles, medical diagnosis, financial fraud detection, and content moderation. These systems are vulnerable to **adversarial examples**—carefully crafted inputs that cause the model to make incorrect predictions.

For a deployed model, a regulator or auditor may ask:

> "Can you prove this model is robust to small adversarial perturbations?"

The model owner may answer:

> "Only if we can keep our model weights, test data, and labels private."

This creates a practical research problem. Public robustness certification exposes proprietary artifacts, while private claims cannot be independently verified.

1.2 Clean Accuracy vs. Certified Accuracy

Clean accuracy only says that the model predicts correctly on unmodified test inputs. Certified accuracy is stronger. It says that for a fraction of test inputs, *every* allowed perturbation inside an ε-ball preserves the correct prediction.

Given a classifier f, input x, label y, and perturbation radius ε, a sample is certified robust if:

$$\forall x' \text{ such that } \|x' - x\|_\infty \leq \varepsilon, \quad f(x') = y.$$

The certified accuracy over a test set of size N is:

$$\frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\text{sample } i \text{ is certified robust}].$$

The research goal is to prove a statement such as:

> "The committed model has certified accuracy at least $X\%$ at radius ε on the committed test set."

without revealing the model, test samples, labels, or which samples passed.

1.3 The Trust Dilemma

Current approaches all have serious limitations:

Approach	Problem
Reveal the model and test set	Breaks model IP and may expose sensitive data
Reveal only a robustness score	Auditor must trust the model owner
Reveal per-sample certificates	May leak test-set and model information
Let the company choose a private test set	The company can cherry-pick easy samples
Use ordinary ZK inference proofs	Proves clean predictions, not certified robustness

Table 1: Current approaches to private robustness auditing and their limitations.

1.4 The Idea: Zero-Knowledge Proofs

We propose using **zero-knowledge proofs** (ZKPs) as a private audit mechanism. The prover produces one proof of the following form:

> "I know private model weights, private test samples, private labels, and valid robustness certificates such that at least T out of N samples are certified robust at radius ε."

The verifier learns the claim is true, but learns nothing about the hidden weights, images, labels, per-sample bounds, or which samples were certified.

The key point is that the proof does **not** trust the prover's DeepPoly output. DeepPoly is run outside the proof only to generate a candidate certificate. The ZK proof checks that this certificate is mathematically valid.

2 Background

2.1 Adversarial Robustness

Given a neural network classifier $f : \mathbb{R}^n \to \{1, \ldots, k\}$ and an input x with true label y, an **adversarial example** is a perturbed input $x' = x + \delta$ such that:

1. $f(x') \neq y$ (misclassification)
2. $\|\delta\|_p \leq \varepsilon$ for some small ε (imperceptibility)

The most common norm is L_∞: $\|\delta\|_\infty = \max_i |\delta_i| \leq \varepsilon$.

Attack methods such as FGSM, PGD, and C&W search for one concrete adversarial input. Robustness certification asks the opposite question: can we prove that *no* adversarial input exists inside the allowed perturbation set?

2.2 DeepPoly-Style Robustness Certificates

DeepPoly is an abstract-interpretation method for neural-network certification. Instead of enumerating all perturbed inputs, it propagates lower and upper bounds through the network.

For each neuron z_i, a certificate contains bounds:

$$l_i \leq z_i \leq u_i.$$

At the output layer, the sample is certified if the lower bound of the true class beats the upper bound of every wrong class:

$$l_{\text{true}} > u_j \quad \forall j \neq \text{true}.$$

DeepPoly handles ReLU layers using cases:

- always active: $l \geq 0$
- always inactive: $u \leq 0$
- mixed: $l < 0 < u$, using a linear relaxation

The proposed system treats these bounds as a **certificate**. The expensive certifier runs offline; the ZK relation only checks that the certificate is consistent with the committed model and input.

2.3 Zero-Knowledge Proofs

A **zero-knowledge proof** is a cryptographic protocol where a prover convinces a verifier that a statement is true, without revealing any information beyond the validity of the statement.

Formally, a ZKP for a relation R allows the prover to prove knowledge of a **witness** w such that $(x, w) \in R$, where x is a public statement.

Properties:

- **Completeness**: If the prover knows a valid witness, they can always convince the verifier
- **Soundness**: If no valid witness exists, the prover cannot convince the verifier (except with negligible probability)
- **Zero-knowledge**: The verifier learns nothing beyond the fact that a valid witness exists

2.4 Commitments, Range Proofs, and Folding

This project needs several cryptographic ingredients:

- **Commitments**: bind the prover to private weights and private test data without revealing them.
- **Range proofs**: prove hidden inequalities such as $l \geq 0$, $u \leq 0$, and $l_{\text{true}} - u_j > 0$.
- **Folding**: aggregate many per-sample checks into one final proof.

Bulletproofs are relevant for hidden range checks, similar to how confidential transaction systems prove hidden amounts are in a valid range. Nova-style folding is relevant because a test set may contain thousands of samples; folding lets the verifier check one accumulated proof instead of thousands of separate proofs.

3 Research Direction

3.1 Core Idea

Design a ZKP system for the following relation.

Public inputs (known to both parties):

- C_W: commitment to model weights

- C_D: commitment to the test set and labels
- ε: robustness radius
- T: required number of certified samples
- model architecture and fixed-point precision parameters

Private witness (known only to prover):

- model weights W
- test samples x_i and labels y_i
- DeepPoly-style certificates: neuron bounds, ReLU cases, and final margins

Statement to prove:

> "I know openings of C_W and C_D and valid certificates showing that at least T samples are certified robust at radius ε."

3.2 Protocol Sketch

1. **Commit**: The model owner commits to quantized model weights and to a fixed test set. If the test set is private, it should be regulator-provided or committed before the audit to prevent cherry-picking.
2. **Generate candidate certificates**: For each test sample, run DeepPoly outside the ZK proof to produce candidate layer bounds.
3. **Check certificates in ZK**: The proof verifies that each certificate follows from the committed model and committed sample:
   - input bounds match $x_i \pm \varepsilon$,
   - affine layer bounds are consistent with the committed weights,
   - ReLU cases are valid using hidden sign/range checks,
   - final class margin proves robustness,
   - labels and samples match the committed dataset.
4. **Count certified samples**: The proof increments a hidden counter for each valid sample certificate.
5. **Aggregate**: Nova-style folding aggregates the repeated per-sample relation into a single proof.
6. **Verify**: The verifier checks one proof and learns only whether the certified-count threshold was met.

3.3 Why Check a Certificate Instead of Proving Full Inference?

Naive ZKML would prove every neural-network operation directly. This is expensive for large networks, especially because ReLU comparisons and range checks are costly inside arithmetic circuits.

The proposed approach uses a proof-carrying-verification pattern:

> The expensive verifier produces a certificate; the ZK proof checks the certificate.

The certificate may still be large, and checking dense linear layers is not free. The research question is whether this certificate-checking relation is substantially cheaper than proving full inference or full bound propagation inside ZK.

3.4 Cryptographic Backends

Tool	Use in this project	Caution
Gnark/R1CS	First prototype for the whole certificate checker	Easiest to build, but may be expensive
Bulletproofs / Bulletproofs++	Hidden range, sign, overflow, and margin checks	Helps inequalities; does not solve private matrix multiplication
Dory / IPA arguments	Transparent committed inner-product proofs	Natural trustless baseline, but may not batch full NN layers well
zkMatrix-style protocols	Committed private matrix/matrix multiplication	Better match for linear layers, but integration with DeepPoly and folding must be proven
Nova folding	Aggregates many sample checks into one proof	Reduces verifier work, not prover work

Table 2: Possible cryptographic components and their roles.

3.5 Why This Is Novel

Existing ZKML work focuses on proving inference, testing, fairness, or training claims. This project targets private **certified robustness** using an abstract interpretation certificate checker.

Work	What it proves	Difference from this project
zkCNN / ZEN	Correct neural-network inference	Clean inference, not certified robustness
pvCNN / verifiable evaluation	Private model evaluation or test accuracy	Empirical accuracy, not robustness certificates
FairProof	Confidential fairness certificate	Closest work; different certificate type and not dataset-level certified accuracy
Kaizen	Correct training of a committed model	Training provenance, not robustness auditing
DeepPoly / CROWN	Public robustness certificates	No privacy-preserving proof layer

Table 3: Related work positioning.

The safer novelty claim is:

> *A zero-knowledge checker for DeepPoly-style robustness certificates, aggregated into a private certified-accuracy proof.*

This avoids overclaiming "first ZKML evaluation" while still targeting a clear gap.

3.6 Potential Applications

- **Regulatory audits**: prove a safety threshold without exposing proprietary weights or sensitive test data.
- **Model procurement**: compare vendors using certified-robustness claims without requiring full model disclosure.
- **Private benchmark reporting**: publish certified-accuracy attestations on restricted datasets.
- **Responsible disclosure variant**: prove existence of a hidden adversarial example without revealing the perturbation.

3.7 Main Technical Obstacles

1. **Private matrix-vector products**: checking affine layer bounds requires proving relations like $y = Wx$ where W, x, and y may all be hidden.
2. **Fixed-point soundness**: DeepPoly is naturally real-valued, but ZK systems use finite-field arithmetic. Rounding, overflow, and signed comparisons must be conservative.
3. **ReLU case validation**: the circuit must prove whether a ReLU is active, inactive, or mixed without revealing the bounds.
4. **Dataset trust**: a private test set is meaningful only if it is regulator-provided, jointly committed, or otherwise protected from cherry-picking.
5. **Practical performance**: the system must beat a strong ZK inference baseline after including all range checks, commitment openings, and aggregation costs.

3.8 Minimum Viable Research Plan

1. Implement a one-image fixed-point DeepPoly certificate checker in Gnark/R1CS for a small MNIST MLP.
2. Measure actual constraints for affine layers, ReLU case checks, range checks, and final margins.
3. Compare against a direct ZK inference baseline.
4. Add commitments for weights, test samples, and labels.
5. Add folding to aggregate many image checks into one certified-accuracy proof.
6. Explore Dory or zkMatrix-style protocols only after the baseline relation is sound and measured.

Part II
Transferability of RL-Learned Adversarial Attack Policies

4 Introduction

4.1 Adversarial Examples and Transfer Attacks

Adversarial examples—small perturbations that cause neural networks to misclassify—are a well-studied vulnerability. A rich literature has shown that adversarial examples crafted on one model can often **transfer** to fool different models, even those with fundamentally different architectures.

For example, an adversarial perturbation computed using gradients on a CNN can sometimes fool a Vision Transformer (ViT), despite the completely different internal mechanisms (convolutions vs. self-attention). This phenomenon is called **transfer attack** and has been extensively studied for gradient-based methods.

4.2 RL-Based Adversarial Attacks

A recent line of work proposes using **reinforcement learning** (RL) to learn adversarial attack policies:

- **RLAB** (Sarkar et al., AAAI 2025): Trains a DQN agent to add patch-based distortions to images to fool a ResNet classifier.
- **Adversarial Agents** (Domico et al., CVPR 2026): Trains an RL agent that generates adversarial examples with fewer queries than gradient-based black-box methods.

These methods learn a **policy**—a state-to-action mapping that decides which perturbations to apply based on the current image state and model feedback.

4.3 The Open Question

No existing work has tested whether RL-learned attack policies transfer across different classifier architectures.

This is a fundamental gap. RL learns a *sequential decision policy* from sparse rewards, which is mechanistically different from computing a gradient and applying a single perturbation. Whether this different kind of learned knowledge generalizes across architectures is unknown.

A comprehensive February 2026 benchmark by Wang et al. reviews over 170 transfer-based attack methods across six categories: gradient-based, input transformation, advanced objective function, generation-based (GAN), model-related, and ensemble-based. **Reinforcement learning does not appear as a category.**

5 Research Question

> **Do adversarial perturbation policies learned by a DQN agent on one classifier architecture transfer to other, fundamentally different architectures?**

Concretely:

1. Train a DQN agent to craft adversarial patch perturbations on a simple CNN (surrogate model)
2. Freeze the learned policy (no further training)
3. Test whether the same policy successfully attacks ResNet-18 and ViT-tiny (target models)

If yes: RL discovers architecture-agnostic perturbation strategies → practical black-box threat

If no: RL overfits to architecture-specific features → reveals fundamental limitations of RL-based attacks

6 Why the Answer Is Non-Trivial

6.1 Gradient-Based vs. RL-Based Attacks

Factor	**Gradient-Based**	**RL Policy-Based**
Mechanism	Single perturbation from backpropagation	Sequential decisions from sparse rewards
Information used	Loss landscape curvature	State-action-reward experiences
Search strategy	Local gradient optimization	Exploration vs. exploitation
Transfer rate (CNN→ViT)	20–50% documented	Unknown

Table 4: Comparison of gradient-based and RL-based attack mechanisms.

6.2 Arguments For Transfer

If the RL agent learns general heuristics like:

- "Target high-variance patches"
- "Perturb edges between objects"
- "Focus on the center of the image"

These are properties of *images*, not specific models. Such strategies should transfer.

6.3 Arguments Against Transfer

The RL agent gets reward based on whether *this specific CNN* flips its prediction. It might learn CNN-specific strategies:

- "Perturb patches that activate specific conv1 filters"
- "Stop when confidence drops below 0.3" (but confidence calibration differs across architectures)

If the policy secretly encodes CNN-specific knowledge, it will fail on ResNet/ViT.
Neither outcome is obvious. That is exactly why this is a research question.

7 Related Work

Paper	What it does	Why it's NOT this proposal
Behzadan & Munir, 2017	Transfers adversarial examples between RL *agents* (DQN→A3C)	Studies fooling RL agents, not using RL to fool image classifiers
RLAB, AAAI 2025	RL agent attacks a single ResNet	No cross-architecture transfer tested
Adversarial Agents, CVPR 2026	RL agent learns attack on one model	Tests held-out inputs of *same* model only
Wang et al., arXiv 2026	Reviews 170+ transfer methods	Zero RL-based attacks in the taxonomy

Table 5: Related work and why it does not address our research question.

8 Potential Outcomes

Outcome	Implication
RL transfers well (>60% on ResNet/ViT)	RL discovers universal image vulnerabilities. Practical black-box threat. Defense research must address policy-based attacks.
RL transfers poorly (<30%)	RL overfits to architecture-specific features. Gradient-based transfer is fundamentally different from policy transfer.
Transfers to ResNet but not ViT	CNN and ResNet share convolutional inductive biases; ViT's self-attention is genuinely different.

Table 6: Possible outcomes and their implications.

Any outcome is valuable because the numbers do not currently exist.

9 Open Questions

1. Does the state representation (architecture-agnostic by design) drive transfer, or does the learned policy still overfit?
2. How does RL transfer compare to gradient-based transfer on the same surrogate/target pairs?
3. Can training on an ensemble of architectures improve transfer to unseen architectures?
4. What strategies does the RL agent actually learn? (Interpretability analysis)

10 Target Venues

- ICLR Workshop on Trustworthy ML

- SaTML (IEEE Conference on Secure and Trustworthy Machine Learning)
- AAAI Workshop on Adversarial Machine Learning
- CVPR Workshop on Security in AI

Part III
Summary

11 Comparison of Research Directions

	Direction 1: ZK Proofs	**Direction 2: RL Transfer**
Domain	Cryptography + ML Security	Reinforcement Learning + Adversarial ML
Core question	Can we privately prove certified robustness over a committed test set?	Do RL attack policies generalize across architectures?
Novelty type	ZK-checkable robustness certificates + fixed-point soundness	New empirical measurement
Main deliverable	Certificate-checking ZK prototype + benchmarks	Transfer success rates + analysis
Skills required	ZK circuits, commitments, DeepPoly/CROWN, fixed-point arithmetic	RL (DQN), PyTorch, CNN/ViT training

Table 7: Comparison of the two research directions.

References

Zero-Knowledge Proofs and ZKML

1. Groth, J. "On the size of pairing-based non-interactive arguments." EUROCRYPT 2016.
2. Liu, T., et al. "zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy." CCS 2021.
3. Feng, B., et al. "ZEN: Efficient Zero-Knowledge Proofs for Neural Networks." IACR ePrint 2021.
4. Yadav, C., et al. "FairProof: Confidential and Certifiable Fairness for Neural Networks." ICML 2024.
5. Abbaszadeh, A., et al. "Zero-Knowledge Proofs of Training for Deep Neural Networks." CCS 2024.
6. Kothapalli, A., et al. "Nova: Recursive ZK Arguments from Folding Schemes." CRYPTO 2022.
7. Bünz, B., et al. "Bulletproofs: Short Proofs for Confidential Transactions and More." IEEE S&P 2018.

Adversarial Examples and Robustness Certification

8. Szegedy, C., et al. "Intriguing properties of neural networks." ICLR 2014.
9. Goodfellow, I., Shlens, J., Szegedy, C. "Explaining and harnessing adversarial examples." ICLR 2015.
10. Madry, A., et al. "Towards deep learning models resistant to adversarial attacks." ICLR 2018.
11. Carlini, N., Wagner, D. "Towards evaluating the robustness of neural networks." IEEE S&P 2017.
12. Singh, G., Gehr, T., Püschel, M., Vechev, M. "An Abstract Domain for Certifying Neural Networks." POPL 2019.
13. Wang, S., et al. "Beta-CROWN: Efficient Bound Propagation with Per-neuron Split Constraints for Neural Network Robustness Verification." NeurIPS 2021.

Transfer Attacks

14. Wang, Z., et al. "Devling into Adversarial Transferability on Image Classification: Review, Benchmark, and Evaluation." arXiv:2602.23117, 2026.

RL-Based Adversarial Attacks

15. Sarkar, S., et al. "RLAB: Reinforcement Learning Platform for Adversarial Black-box Attacks." AAAI 2025.
16. Domico, N., et al. "Adversarial Agents: Black-Box Evasion Attacks with Reinforcement Learning." CVPR 2026.
17. Behzadan, V., Munir, A. "Vulnerability of Deep Reinforcement Learning to Policy Induction Attacks." 2017.